UTDT

Tecnología Digital V:
Diseño de Algoritmos

Trabajo Práctico
Seam Carving

Integrantes
Franco Arrieta
Gonzalo Benzaquen
Julieta Kovalivker

Motivación

El redimensionamiento de imágenes es una tarea fundamental en el procesamiento digital de imágenes. El enfoque más sencillo, el recorte, elimina píxeles de los bordes, descartando potencialmente contenido relevante. El escalado uniforme, por su parte, distorsiona las proporciones de los objetos presentes en la imagen. Ante estas limitaciones, Seam Carving [1] propone una técnica de redimensionamiento consciente del contenido (content-aware): en lugar de eliminar filas o columnas enteras, se eliminan costuras (seams), es decir, caminos de píxeles conectados que atraviesan la imagen de un extremo al otro pasando por regiones de baja importancia visual.

Esta técnica fue adoptada en herramientas de edición ampliamente utilizadas, como Adobe Photoshop (bajo el nombre Content-Aware Scale) y GIMP. La idea central es preservar las regiones más salientes de la imagen: caras, objetos, texturas complejas, mientras se contraen las zonas de menor contenido informativo.

El propósito de este trabajo es resolver el problema de Seam Carving mediante tres algoritmos diferentes (Fuerza Bruta, Backtracking y Programación Dinámica) para así comparar el rendimiento de las distintas implementaciones junto a sus complejidades computacionales. Resulta relevante la implementación de diferentes algoritmos para lograr encontrar el más adecuado para el problema a resolver, lo cual no siempre es evidente.

Definición del problema y modelado

Dada una imagen representada como una matriz de n filas y m columnas, donde cada celda (i, j) tiene asociado un valor de energía $e_{i,j} \geq 0$, una costura vertical es una secuencia de píxeles:

$$s = ((1, c_1), (2, c_2), ..., (n, c_n))$$

tal que para todo $i \in \{1, ..., n-1\}$ se cumple $|c_{i+1} - c_i| \leq 1$, es decir, los píxeles consecutivos de la costura son adyacentes vertical o diagonalmente. La energía de la costura se define como

$$E(s) = \sum_{i=1}^{n} e_{i,ci}$$

El objetivo del problema es encontrar la costura vertical de mínima energía.

[1] Seam carving. (n.d.). In Wikipedia. Retrieved April 15, 2026, from https://en.wikipedia.org/wiki/Seam_carving

Para resolver el problema, tomamos la matriz de energía de la imagen y buscamos, a lo largo de cada columna, cuál de ellas permite alcanzar la costura de menor costo total. A partir de esta idea, implementamos una solución utilizando cada uno de los algoritmos mencionados.

Implementación mediante Fuerza Bruta

La idea detrás del algoritmo de Fuerza Bruta es calcular y comparar todos las costuras verticales posibles, guardando en cada iteración la costura de menor energía obtenida hasta el momento. De esta forma, al finalizar la ejecución del algoritmo, se habrán probado y comparado todas las opciones posibles lo que nos garantiza que la costura obtenida sea la de menor energía.

---

calcularEnergia (s)

```
sumaEnergia ← 0
for fila ∈ energia then
        sumaEnergia += energia[fila][s[fila]]
end for
return sumaEnergia
```

FB (s, x, y)

```
if esUltimaFila(x) ∧ esColumnaValida(y) then
        if calcularEnergia(s) < calcularEnergia(best) then
                best ← s
        end if
end if
else if esColumnaValida(y) then
        s' ← s ∪ y
        FB(s', x + 1, y)                // BAJO VERTICAL
        FB(s', x + 1, y - 1)            // BAJO A LA IZQUIERDA
```

```
        FB(s', x + 1, y + 1)            // BAJO A LA DERECHA
end if
```

EncontrarSeamFB (energia)

```
best ← ∅
for y ∈ {0, …, m-1} do
        s ← ∅
        FB(s, 0, y)
end for
return best
```

Implementación mediante Backtracking

Para la implementación de Backtracking, extendemos la idea de Fuerza Bruta incluyendo una poda. En Fuerza Bruta nos encontramos computando casos que, en base a la solución parcial, sabemos que podrían ser descartados.
Teniendo guardada la mejor solución encontrada hasta el momento, es posible compararla con la que se está computando en cada iteración. Así, en el caso donde nos encontramos en una costura vertical (no necesariamente completa) con mayor energía que la solución parcial, sabemos que esta puede ser descartada ya que existe al menos una costura vertical que será más óptima.

BT (s, x, y)

```
if esUltimaFila(x) ∧ esColumnaValida(y) then
        if calcularEnergia(s) < calcularEnergia(best) then
                best ← s
        end if
end if
else if esColumnaValida(y) ∧ calcularEnergia(best) ≥ calcularEnergia(s ∪ y) then
        s' ← s ∪ y
        BT(s', x + 1, y)                // BAJO VERTICAL
        BT(s', x + 1, y - 1)            // BAJO A LA IZQUIERDA
        BT(s', x + 1, y + 1)            // BAJO A LA DERECHA
end if
```

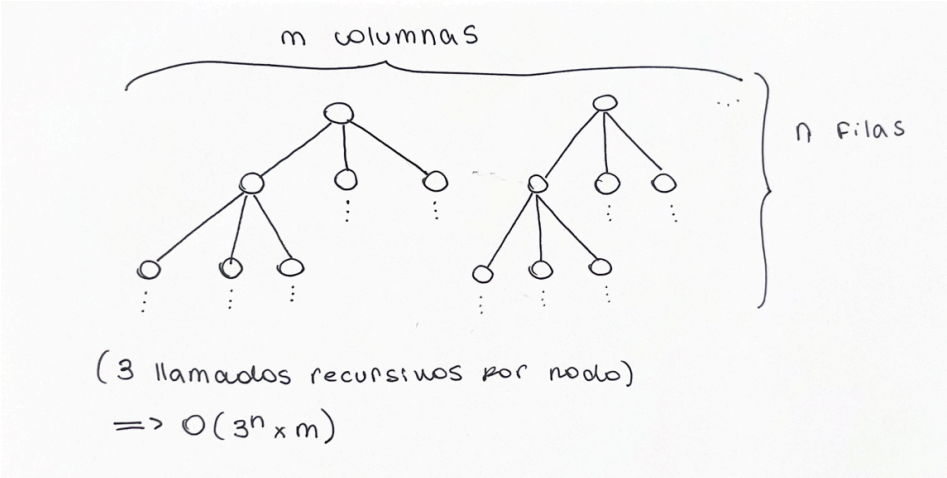
EncontrarSeamBT (energia)

```
best ← ∅
for y ∈ {0, …, m-1} do
```

s ← ∅
BT(s, 0, y)
end for
return best

Complejidad en Fuerza Bruta y Backtracking:
Como la complejidad temporal contempla el peor caso, tanto Fuerza Bruta, como Backtracking tendrán la misma complejidad. Esto se debe a que en el peor caso, la imagen nunca cumplirá la condición de la poda.

Así, dado $n = \#filas$ y $m = \#columnas$:
Complejidad Temporal
Dado un nodo de la primera fila se llamará a la recursión tres veces (abajo y diagonales inferiores). Cada uno de ellos volverá a llamar tres recursiones hasta llegar a la última fila, n. Es decir, la complejidad de calcular la mínima energía de una columna vertical es $O(3^n)$. Cómo queremos hacerlo con la imagen entera, lo computamos por cada nodo en la primera fila, es decir, m veces.
Por lo tanto, la complejidad temporal total de ambos algoritmos es de $O(3^n * m)$.



Complejidad Espacial
En ambos algoritmos se necesita memoria para crear dos vectores de tamaño n. Luego, la complejidad espacial es $O(2n)$ que es igual a $O(n)$.

Implementación mediante Programación Dinámica

En ambas implementaciones anteriores, notamos que muchas costuras verticales se superponen. En esos casos la parte superpuesta de la costura es calculada reiteradas veces. Esto nos da un indicio de que guardarnos cálculos parciales permitiría ahorrarnos recomputarlos en costuras

posteriores que presentan superposiciones. La solución que planteamos fue pensada con un enfoque *top-down*.
Cuando nos encontremos con una superposición, el problema de calcular la mejor costura vertical a partir de ese vértice será igual para ambos. La solución al problema es única, por lo tanto, ambos llegarán a la misma costura vertical parcial.

Cuando hablamos de guardar cálculos parciales nos referimos a la implementación de una estructura *memo*. Nos interesará poder guardar la solución parcial para cada nodo. Así, cuando queramos calcular una costura vertical repetida, la podremos obtener del memo. Cada posición tendrá guardada la segunda coordenada correspondiente al siguiente nodo necesario para el armado de la mejor costura vertical desde sí mismo. Al terminar la ejecución, podremos recuperar la solución óptima tomando a los "hijos" de cada nodo.
En nuestra implementación, para evitar la mayor cantidad de cálculos repetidos, no solo guardamos el siguiente nodo de la mejor costura vertical, sino que también guardamos la suma de energía que esta posee. Para la simplificación del pseudocódigo, no la incluimos.

PD (x, y, memo)

```
if esColumnaValida(y) then
        if memo[x][y] != null then
                return memo[x][y]
        end if
        else if esUltimaFila(x) then
                memo[x][y] ← -1
                return memo[x][y]
        end if
        else
                vertical ←  PD(x + 1, y, memo)                    // BAJO VERTICAL
                izquierda ← PD(x + 1, y - 1, memo)                // BAJO A LA IZQUIERDA
                derecha ← PD(x + 1, y + 1, memo)                  // BAJO A LA DERECHA
                min ← minCarving(vertical, izquierda, derecha, y)
                memo[x][y] ← min
                return  memo[x][y]
        end if
end if
return ∅
```

---

minCarving (vertical, izquierda, derecha, y)

---

min3 ← min(calcularEnergia(vertical), calcularEnergia(izquierda), calcularEnergia(derecha))

```
if calcularEnergia(vertical) == min3 then return y - 1 end if
if calcularEnergia(izquierda) == min3 then return y + 1 end if
return y,
```

---

reconstruir (memo, best, n)

```
res ← {best}
prev ← best
for x ∈ {0, …, n-1} do
        res ←res ∪ memo[x][prev]
        prev ← memo[x][prev]
end for
return res
```

EncontrarSeamPD (energia)

```
best ← -1
for y ∈ {0, …, m-1} do
        s ← PD(0, y, memo)
        if calcularEnergia(s) < calcularEnergia(best) then
                best ← s
        end if
end for
return reconstruir(memo, best, n)
```

Complejidad Temporal

Al igual que en las funciones anteriores, Programación Dinámica se encarga de llamar a la recursión tres veces por cada nodo en cada columna comenzando desde la primera fila hasta llegar a la última. A diferencia de ellos, utilizara la estructura memo para asignar, indirectamente (con guardar solamente el hijo indicado bastará, ya que luego podrá ser reconstruido), la mínima costura vertical desde cada vértice hasta el final de la matriz, lo cual evitará la repetición de cálculos en búsqueda de las costuras verticales mínimas. De este modo, la complejidad temporal será calcular la costura vertical mínima de cada nodo una vez. Como la matriz cuenta con $n * m$ nodos, su complejidad será $O(n * m)$.

Complejidad Espacial

En el caso de la implementación con Programación Dinámica, es necesario generar y mantener la estructura memo. Esta estructura será una matriz de $n * m$ donde, dada una fila i e una columna j se guardará un entero que nos dirá en qué columna continua la costura vertical óptima y un

double que nos dirá cuánta energía suma la respectiva costura vertical. Luego, por cada columna, la cantidad de elementos que se guardarán será:

$$\sum_{i=1}^{n} 2 = 2 * n$$

En complejidad espacial:

$$O(2 * n) = 2 * O(n) = O(n)$$

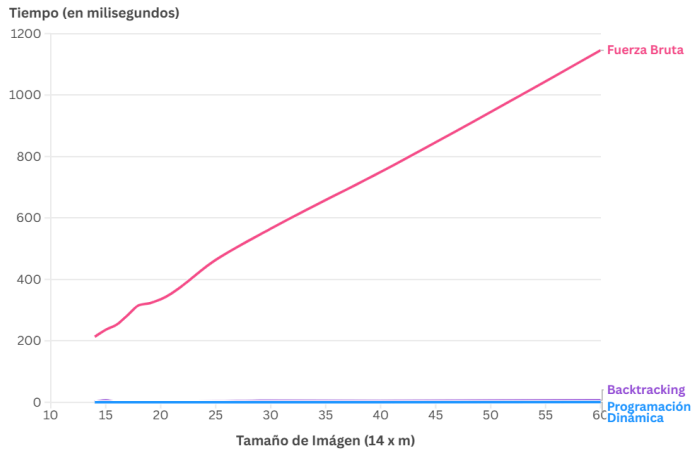
Como la matriz cuenta con m columnas, la complejidad espacial total de la función será $O(n * m)$.

Experimentación

Para garantizar la consistencia, todos los tests se realizaron en la misma computadora, ejecutando una máquina virtual de Linux.

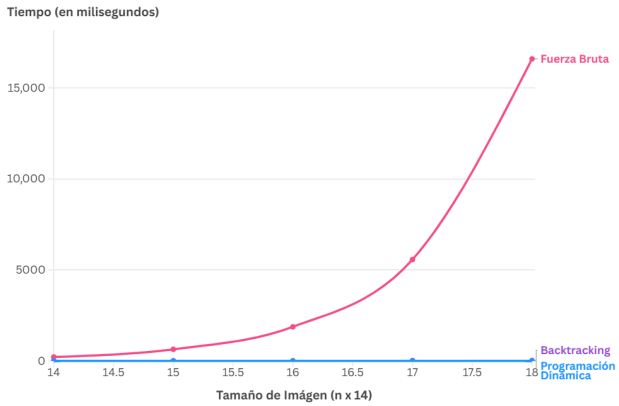
Comparación sobre tamaños

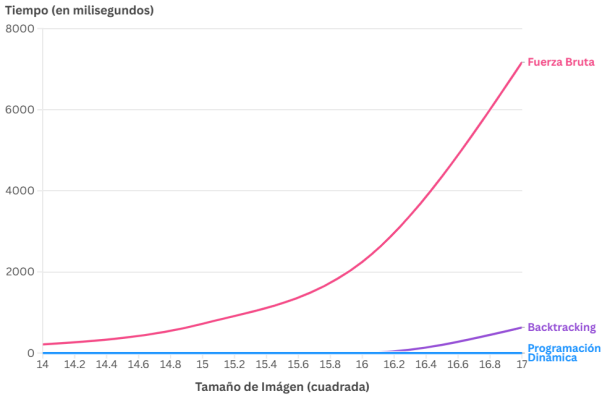
Para este apartado decidimos utilizar diferentes tamaños de imágenes para evaluar el comportamiento de los algoritmos con las mismas. En primer lugar queríamos ver cómo se comparaban los tiempos de ejecución. Para ello hicimos dos pruebas. Por un lado modificando m (la cantidad de columnas) y, por otro lado, n (la cantidad de filas). En cuanto a las imágenes, decidimos usar la misma e ir modificando su tamaño, cosa que todos los tamaños resulten en imágenes similares.



En este gráfico se puede observar la variación en el tiempo de ejecución de cada algoritmo al modificar m. Es evidente que la complejidad de Fuerza Bruta se comporta de forma lineal en función de m.

El siguiente gráfico es similar solo que la modificación es sobre n. Se ve que, al aumentar su valor, hay un crecimiento exponencial sobre la complejidad.
Por otro lado, también decidimos hacer un test sobre imágenes cuadradas, donde el n y el m aumentan por igual.

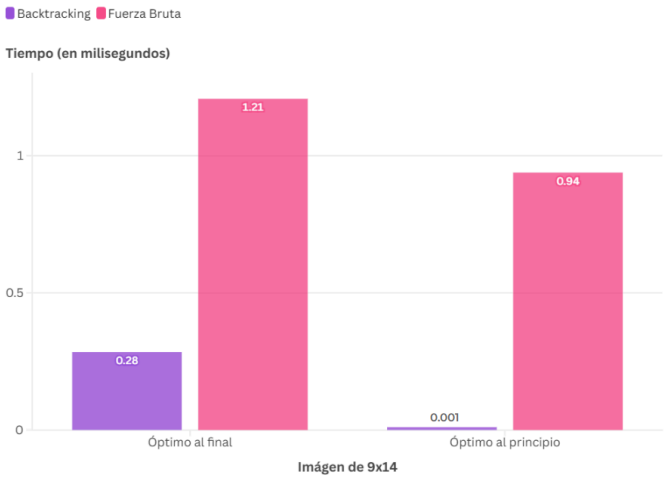




En las tres pruebas, pudimos notar cómo en el algoritmo Fuerza bruta, al aumentar el tamaño del input, sufre de mayor crecimiento en su tiempo de ejecución. Esto se debe a la propia complejidad temporal del algoritmo, el cual depende de n de forma exponencial y de m de forma lineal. Backtracking posee la misma complejidad que Fuerza Bruta pero gracias a la implementación de las podas, con inputs pequeños como los que usamos para los tests, las variaciones en el tiempo de ejecución son casi imperceptibles. Con un mayor número de iteraciones, creemos que la forma de la función será cada vez más parecida a la de Fuerza Bruta. La complejidad de Programación Dinámica, a diferencia de los dos anteriores, depende de forma lineal tanto de n como de m, también necesitaremos un mayor tamaño de input para notar el crecimiento de la función.

Comparación sobre contenido del input

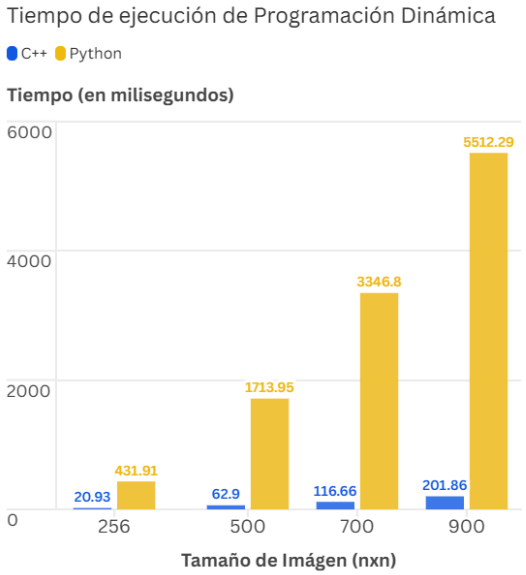
Para este apartado, quisimos poner a prueba la capacidad de Backtracking y ver si, en la práctica, es más eficiente que Fuerza Bruta. Como ya mencionamos, estos dos tienen la misma complejidad, pero Backtracking logra evitar ciertos cálculos. Si logramos encontrarnos con una imagen que posea una distribución de energía, tal que se puedan realizar podas en muchas iteraciones.
Para testear esto, se nos ocurrió probar un input que posea menor energía del



lado izquierdo y otro del lado derecho. Por cómo se ejecuta nuestra implementación, cuando el óptimo se encuentre más a la izquierda, este se encontrará primero y luego se podrán realizar más podas. Este gráfico de barras compara el tiempo de ejecución de Backtracking con Fuerza Bruta en ambos casos. Es evidente que, cuando el óptimo se encuentra al principio, hay una mayor mejora temporal comparado con Fuerza Bruta que cuando está más al final. Igualmente, esto no significa que cuando el óptimo esté al final no se realizan podas, sino que podremos podar más seguido con el óptimo al principio.

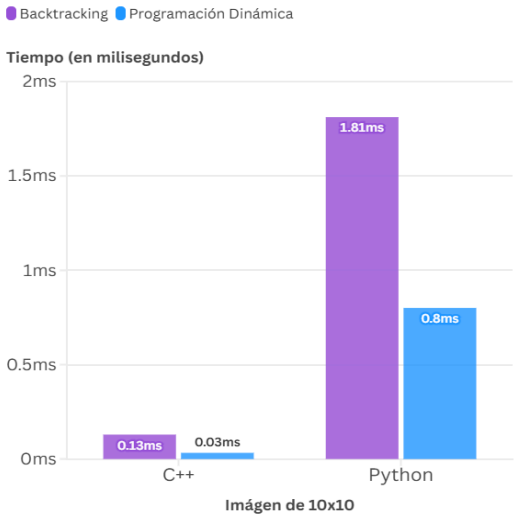
Comparación entre la implementación en C++ y Python

Para esta sección implementamos los algoritmos tanto en C++ como en Python, intentando que sean lo más similares posibles en ambos lenguajes. La traducción a Python fue realizada con la herramienta de inteligencia artificial Claude, indicando que no modifique la funcionalidad del mismo. De todos modos, leímos atentamente su implementación para asegurarnos de que sea lo más fiel posible al código en C++. Dejamos el link a la conversación en la cuál se realizó la traducción.

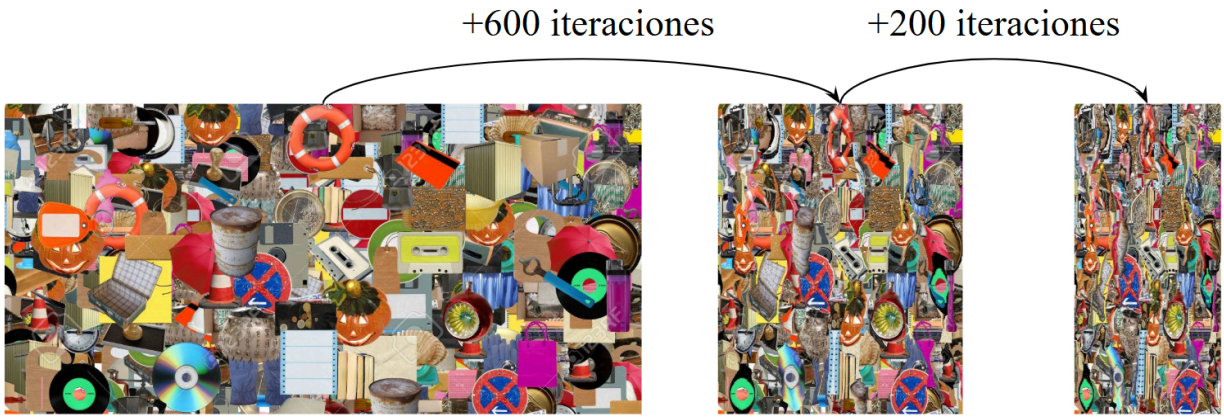


Nuestra hipótesis era que la ejecución del algoritmo en C++ sea significativamente más rápida que su equivalente en Python, debido a que C++ es un lenguaje compilado de bajo nivel, mientras que Python es un lenguaje interpretado con tipado dinámico. En el gráfico pareciera que nuestra hipótesis es acertada y que, de hecho, el algoritmo de Python es bastante más lento y su tiempo de ejecución crece mucho más sobre el tamaño de la imagen. Realizamos el test solo sobre Programación Dinámica ya que las otras implementaciones eran muy lentas para estos tamaños de input.

Para poder ver si la performance de Backtracking tiene tiempos de ejecución menores a Fuerza Bruta, probamos con una imagen más pequeña y comparamos tiempos entre ambos algoritmos tanto con Python como con C++.
Los resultados en este test también fueron los esperados. Backtracking tarda significativamente más en ejecutarse en Python que en C++.



Imágen con mucha información
Se nos ocurrió que podríamos ver qué pasa si aplicamos Seam Carving con una imagen sin un recorte evidente. Nuestra hipótesis es que se va a deformar mucho ya que los píxeles de la imágen son todos significativos, mientras más iteraciones se corran, más se distorsionará.



m = 1300 (imagen original) m = 700 m = 500

Luego de ejecutar el algoritmo, podemos notar cómo con el aumento en las iteraciones, la imagen se distorsiona cada vez más. Siendo así, muy difícil identificar los objetos presentes en la original. Cabe destacar que el número de iteraciones necesarias para notar una distorsión significativa, fue mayor al que esperábamos.

Dificultades encontradas

Podas en Backtracking
A la hora de pensar podas para incluir en el algoritmo de Backtracking, notamos que la más "relevante" ya la habíamos incluido en Fuerza Bruta, o sea, la poda de factibilidad. Al hacer Fuerza Bruta, podríamos haber optado por calcular todas las costuras verticales incluyendo las que ni siquiera están conectadas entre sí, para luego quedarnos con las factibles. Haber hecho esta implementación hubiese resultado en un algoritmo menos eficiente. Es por eso, que decidimos dejar la poda de factibilidad en Fuerza Bruta (aunque al ser tan natural, en un principio no la pensamos como una poda). A partir de esto, surge la duda, ¿qué otra poda se podría incluir en Backtracking? Eventualmente logramos resolverlo con la poda de optimalidad ya mencionada en el apartado de Backtracking.

Optimización de complejidad espacial en Programación Dinámica
Al idear la estructura memo, nuestra primera intuición fue armar una matriz que guarde, por nodo, un vector con los nodos de la mejor costura vertical desde sí mismo. A pesar de ser una estructura cómoda, notamos que la complejidad espacial era muy alta y quisimos reducirla. Como el memo se trataba una matriz de $n * m$ donde, dada una fila i e una columna j se tenía

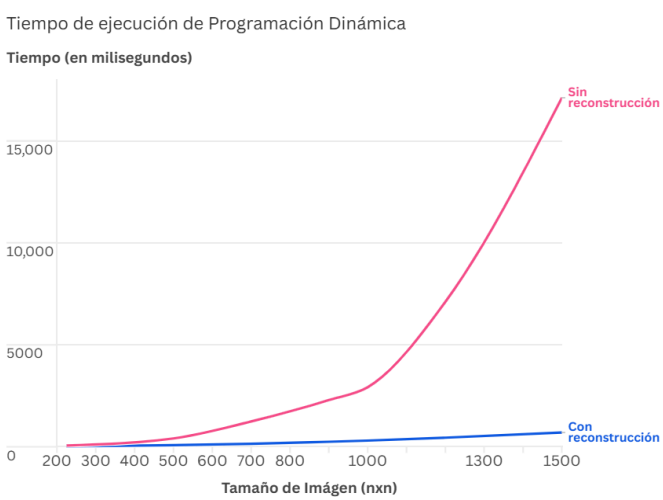
un vector v de tamaño $|v| = n - i$, para todo $i \in \{0, ..., n-1\}$, por cada columna la cantidad de elementos que se guardaban era:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$$

En complejidad espacial:

$$O(\tfrac{n^2+n}{2}) = O(n^2+n) = O(n^2)$$

Como la matriz contaba con m columnas, la complejidad espacial del memo original era $O(n^2 * m)$.
Dada la alta complejidad espacial de este enfoque, decidimos buscar otra implementación. Así, como segundo intento modificamos el enfoque en la estructura memo para obtener una complejidad menor tal y como se explicó en el apartado de Programación Dinámica.



Para verificar la mejora en la implementación de Programación Dinámica, decidimos comparar los tiempos de ejecución de ambas versiones. Es muy evidente que evitar la copia de vectores en la actualización del memo mejora significativamente el rendimiento del programa.

Optimización del cálculo de energía en Programación Dinámica
Por otra parte, al optimizar la estructura memo notamos que era ineficiente recalcular la energía de cada costura vertical desde cada nodo. Para resolverlo, pensamos en dos alternativas. Por un lado en cada posición del memo, incluir junto con el siguiente nodo del camino mínimo, la energía del mismo (guardado en un pair). La otra alternativa, sería mantener dos estructuras memo, donde una guarda en i, j su siguiente nodo (de los tres posibles) en el camino mínimo y la otra, guarda la energía mínima de la mejor costura vertical desde el nodo en esa posición. Aunque la complejidad espacial no cambia (en ambos casos guardamos un double más por cada nodo), decidimos implementar la primera ya que nos pareció más prolija y fácil de implementar.

Tamaño de las imágenes
En cuanto a las imágenes, para poder mostrar las diferencias entre las ejecuciones de los algoritmos Backtracking y Fuerza Bruta tuvimos que utilizar tamaños de hasta 18x18 en C++ ya que un tamaño mayor no permitía obtener un resultado en un tiempo razonable.

Conclusión

Posibles Mejoras
En nuestra experimentación nos limitamos a testear sobre pocas imágenes (en varios casos, una única) por una cuestión de tiempos de ejecución. Así como posible mejora, se podría incluir tests que en lugar de comparar rendimiento sobre una única imagen, lo realice sobre el rendimiento promedio de varias. De esta forma, tener en cuenta más casos y disminuir el desvío estándar.

En base a nuestras observaciones pudimos notar una gran diferencia en los resultados de las distintas implementaciones, donde los algoritmos Backtracking y Fuerza Bruta podían ejecutarse con imágenes mucho más chicas (18x18) que Programación dinámica (2000x2000). Esta cota la encontramos aumentando el tamaño del input, como se aprecia en los gráficos.

Tras realizar nuestros experimentos notamos que nuestras observaciones son coherentes con el análisis de complejidad de cada implementación. Luego, concluimos que es muy ineficiente resolver el problema con Fuerza Bruta y Backtracking, en cambio la Programación Dinámica muestra ser más versátil en términos de tamaño de input y tiempo de ejecución.